



XRPL Hackathon @ XRP Australia

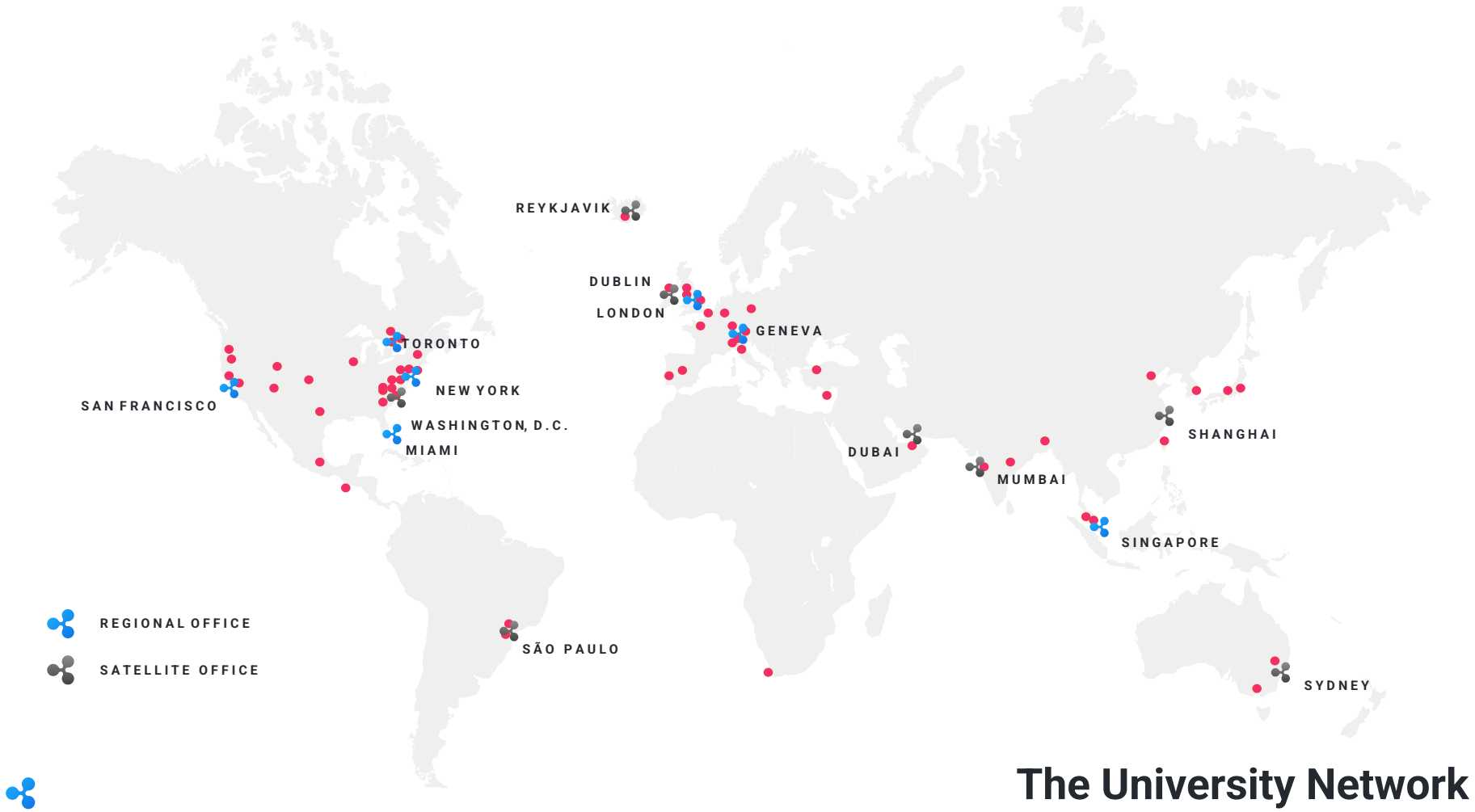
2026

University Blockchain Research Initiative

62

Universities worldwide focused on
accelerating research and development





The University Network

Tangible Impact

UBRI Results

1,550

Research projects

803

Fintech courses

1567

Fellowships/
Scholarships

1300

Events

Ripple Results

28

XRPL Grant
Winners

356

Projects on XRPL

91

Hires

61k

Podcast
Subscribers



UBRI Creates Network Effects

7 Global Conferences

1,400+ attendees

5 Research Highlights Reports



UBRI Research Corpus

Live on xrpl-commons.org/learn/ubri-research-database



Joseph Chen

 Commons

[Learn](#) [Build](#) [Launch](#) [Adopt](#) [Engage](#) [Sustainability](#) [About Us](#) [Contact](#)

UBRI Research Corpus

Discover academic research on Blockchain and its impact across diverse sectors. The UBRI Research Corpus connects you with publications from top universities worldwide.



Select Theme:

All

Select University:

All

Select Year:

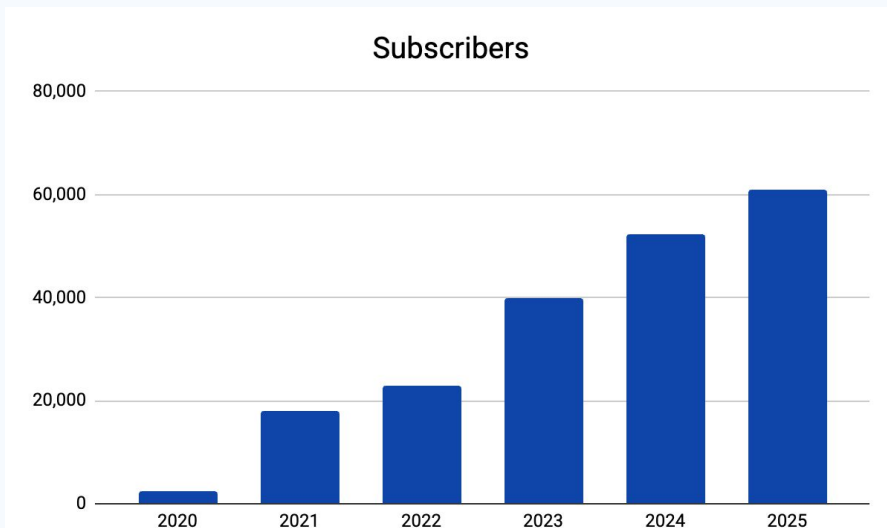
All

Analyze

© 2024 Ripple UBRI

Award Winning Podcast

- 7 Seasons, 54 Episodes
- 62K Subscribers
- Global audience: 54% Americas, 24% APAC, 22% EMEA
- Ranked in the **Top 25% of all podcasts worldwide**



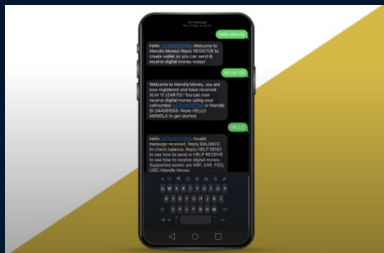
Hackathon to Enterprise Adoption

Zero to Hero Lifecycle Winners

Julian Kanjere



PhD Student
University of Cape Town



Mandla Money wins XRPL
Creating Real World Impact
Hackathon



Mandla Money awarded
XRPL Grant to Advance
Financial Inclusion



Mandla Money at Apex
Developer Conference in Las
Vegas



● Internal

XRPL Student Builder Residency

A 4-week accelerated learning experience for exceptional students in technical disciplines interested in building on the XRP Ledger.

Students built individual XRPL projects with mentorship from expert XRPL core contributors, developers, and community members.



Meet Your Mentors

2026



Hackathon workshop



Joel Mun

Senior Developer Relations,
Ripple

[x.com/9oelM](https://twitter.com/9oelM)

Challenge Statement

Build an **MVP** that leverages the **XRP Ledger's** core features to solve a **real-world problem**.

- Experimentation with a combination of **XRPL's features** like
 - Lending protocol & Single asset vault
 - Smart escrows
 - Multi-purpose tokens
 - Batch transaction
 - TokenEscrow
- **SDKs** for other developers to easily integrate XRPL features into their apps
- **Payment apps** leveraging stablecoins, microfinance, tokenization
- **Stablecoin use cases** like AUDD or RLUSD

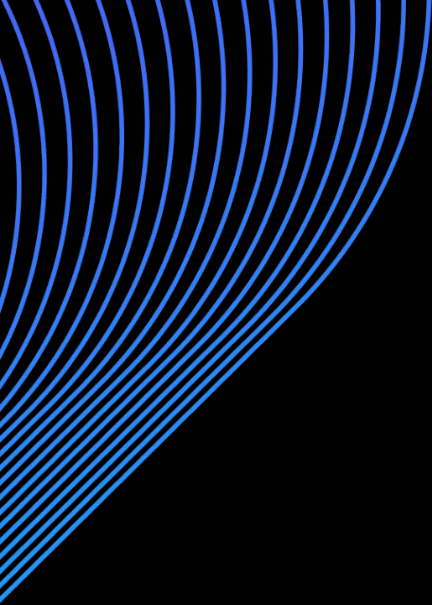
Prize Pool

- **1st:** 3,000 AUD
- **2nd:** 2,000 AUD
- **3rd:** 1,000 AUD
- **Best developer feedback:** 250 AUD



<https://shorturl.at/fMpqT>

Submit developer
feedback here



How XRPL works

The XRPL way

Others

**Write, test,
audit,
maintain,
upgrade smart
contracts from
scratch**

XRPL

**Everything
built-in;
No smart
contract
overhead**

Everything built-in.. how?

Amendments

<https://livenet.xrpl.org/amendments>

<https://github.com/XRPLF/XRPL-Standards>

Featured amendments

● Live on XRPL Mainnet

Batch

Multiple transactions in a bundle

● Live on XRPL Mainnet

Multi-purpose token (MPT)

Fungible token standard optimized for tokenization.

● Live on XRPL Mainnet

Token escrow

Support escrowing issued tokens (IOU) or MPTs.

● Live on dedicated devnet

Smart escrow

Programmable escrow with flexible, on-chain release conditions

● Live on XRPL devnet

Lending protocol

Undercollateralized loans using pooled funds from a single asset vault.

Smart contracts

<https://livenet.xrpl.org/amendments>

Other amendments/features

Automated Market Maker

Provide liquidity, earn yield, and trade tokens via an AMM

Issued Token (IOU)

Ability to represent fungible tokens

NFT

Implements non-fungible tokens with built-in royalties where all trades handled by the DEX

Token Asset Controls

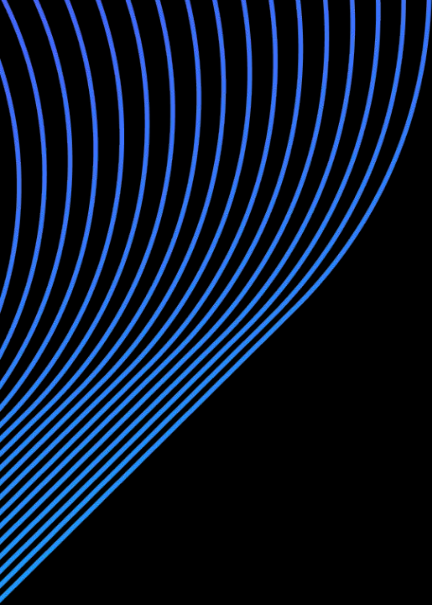
Controls for token issuers and holders for better security and compliance

Advanced Payments

Payment capabilities like “Payment channels” and “Checks”

Permissioned DEX

Controlled environments for trading within the XRPL's DEX.



XRPL Quickstart



<https://github.com/9oelM/xrp-aus-2026-workshop>


```
1  export async function step1ConnectAndFundWallet
   (): Promise<Wallet> {
2    const wallet = Wallet.generate()
3    console.log(`[Step 1] Generated wallet address:
   ${wallet.address}`)
4    console.log('[Step 1] Requesting funds from testnet faucet...')
5
6    const response = await fetch(FAUCET_URL, {
7      method: 'POST',
8      headers: { 'Content-Type': 'application/json' },
9      body: JSON.stringify({ destination: wallet.address,
   userAgent: 'xrpl-workshop' }),
10   })
11
```

1. Create & fund wallet

```
1 export async function step2WaitForFunding(  
2   client: Client,  
3   address: string,  
4   timeoutMs = 60_000,  
5 ): Promise<number> {  
6   console.log(`[Step 2] Waiting for ${address} to be funded...`)  
7   const deadline = Date.now() + timeoutMs  
8  
9   while (Date.now() < deadline) {  
10    try {  
11      const balance = await client.getXrpBalance(address)  
12      if (balance > 0) {  
13        console.log(`[Step 2] Balance confirmed: ${balance} XRP`)  
14        return balance  
15      }  
16    } catch {  
17      // Account not yet visible on the ledger – keep polling  
18    }  
19    await sleep(POLL_INTERVAL_MS)  
20  }  
21 }
```

2. Wait for the wallet to be funded

```

1  export async function step3SendPayment(
2    client: Client,
3    senderWallet: Wallet,
4    destinationAddress: string,
5    amountDrops: string,
6  ): Promise<string> {
7    console.log(
8      `[Step 3] Sending ${dropsToXrp(amountDrops)} XRP
→ ${destinationAddress}`,
9    )
10
11    const payment: Payment = {
12      TransactionType: 'Payment',
13      Account: senderWallet.address,
14      Destination: destinationAddress,
15      Amount: amountDrops,
16    }
17
18    const prepared = await client.autofill(payment)
19    const { tx_blob, hash } = senderWallet.sign(prepare
d)
20
21    await client.submit(tx_blob)

```

3. Send payment tx

```
1 export async function step4WaitForConfirmation(  
2   client: Client,  
3   txHash: string,  
4   timeoutMs = 60_000,  
5 ): Promise<ValidatedTxResult> {  
6   console.log(`[Step 4] Waiting for tx ${txHash} to be validated...`)  
7   const deadline = Date.now() + timeoutMs  
8  
9   while (Date.now() < deadline) {  
10    try {  
11      const response = (await client.request({  
12        command: 'tx',  
13        transaction: txHash,  
14        binary: false,  
15      })) as unknown as ValidatedTxResult  
16  
17      if (response.result.validated) {  
18        const txResult = response.result.meta.TransactionResult  
19        console.log(`[Step 4] Confirmed on ledger. Result: ${txResult}`)  
20        return response  
21      }  
22    } catch {  
23      // Transaction not yet in a validated ledger – keep polling  
24    }  
25    await sleep(POLL_INTERVAL_MS)  
26  }  
27 }
```

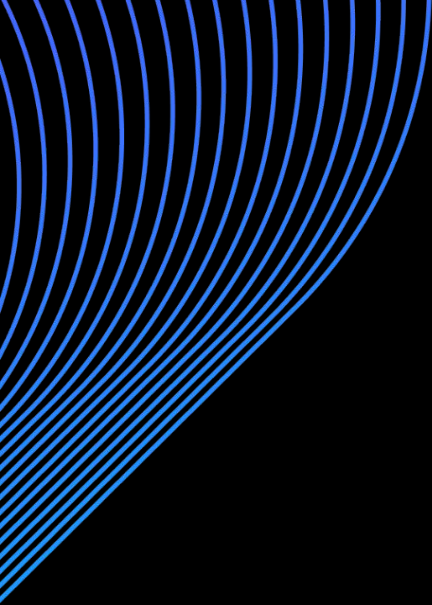
4. Wait for tx confirmation

Payment Success

HASH:35A29558760BDCAC16B4561FE855A4060B1E60757A2B2FBC572699E096F8494C

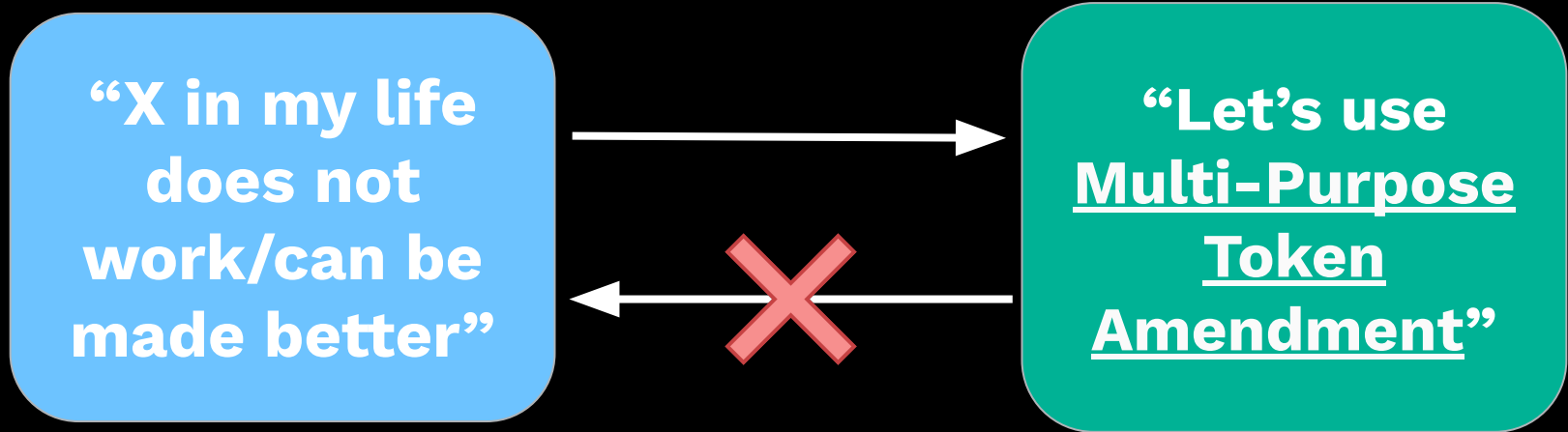
CTID:C0E8C92300040001

Check your
payment tx on
the explorer

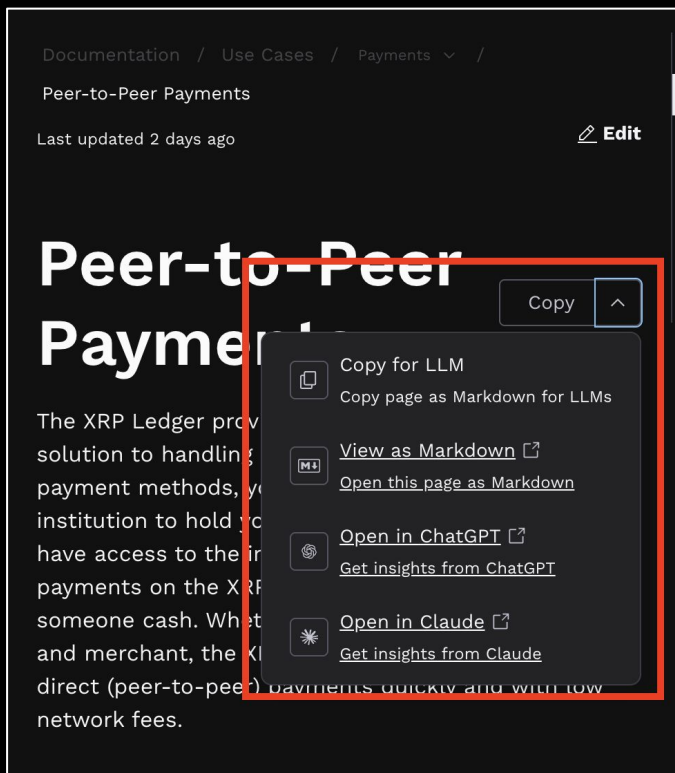


Tips

Tip #1: Start with a problem, not an XRPL feature



Tip #2: Use AI more efficiently



‘Copy for LLM’ on xrpl.org/docs

OR

Access markdown docs on GH
directly:
github.com/XRPLF/xrpl-dev-portal/tree/master/docs

Tip #3: Stick to the judging criteria

- **Innovation:** a novel idea, not just a rehash of existing solutions.
- **Technical Execution:** working, stable code that demonstrates genuine engineering effort by leveraging the features of XRPL
- **Real-World Impact:** solve a real problem for real people
- **Presentation Quality:** a clear, confident pitch that demonstrates the main idea

Some more tips

- Scope: not too small, not too big
- Demo: do your best to show one although it's not a must
- Luma: Refer to [Luma page instructions](#) when lost



Newest amendment references

Smart escrows

Smart escrows are still on devnet phase. You need to use a specific set of tools and endpoints to use it. You will need to code in Rust. If you are looking for more challenge, this might be for you.

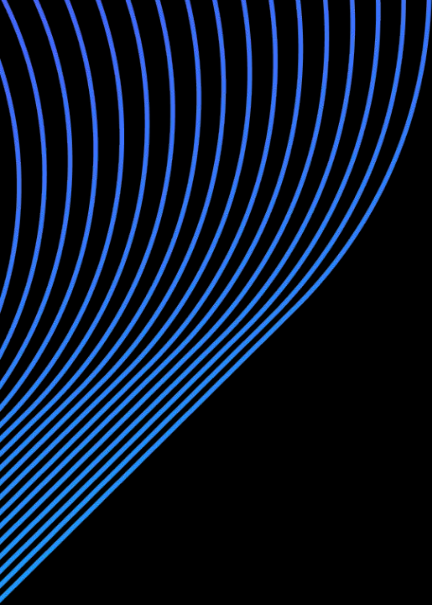
- [Amendment reference](#)
- [Use cases](#)
- API endpoint: wasm.devnet.ripple.test.net (port 51234 for HTTP, 51233 for WS)
- Smart escrows explainer: [#1](#), [#2](#), [#3](#)
- Explorer: <https://custom.xrpl.org/wasm.devnet.ripple.test.net>
- Faucet: <https://wasmfaucet.devnet.ripple.test.net/accounts> (send a POST request)
- Rust (wasm) extension: <https://crates.io/crates/xrpl-wasm-stdlib@v0.8.0>
- Rust extension testing UI: <https://ripple.github.io/xrpl-wasm-stdlib/ui/>
- Rust extension examples:
<https://github.com/ripple/xrpl-wasm-stdlib/tree/main/examples/smart-escrows>
- Python version: [xrpl-py@v4.5.0b0](#)
- JS: [xrpl@4.6.0-smartescrow.0](#), [ripple-binary-codec@2.7.0-smartescrow.0](#) (you can also use the @ smart-escrow-experimental tag)

Lending protocol & Single asset vault

- [Amendment reference](#)
- [Lending protocol explainer](#)
- Environment: [devnet](#) (!= testnet)
- Python version: [xrpl-py@v4.5.0b0](#)
- JS: [xrpl@4.6.0-smartescrow.0](#), [ripple-binary-codec@2.7.0-smartescrow.0](#) (you can also use the @ smart-escrow-experimental tag)

Other amendments

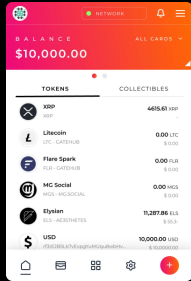
- [Multi-purpose token explainer](#)
- [Batch transaction explainer](#)



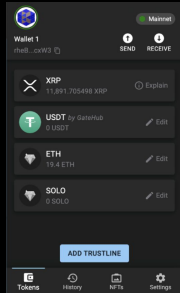
Appendix (resources)

For the demo, consider using these wallets:

Browser



crossmark.io



gemwallet.app

Mobile



joeywallet.xyz



xaman.app

and more!

How to get RLUSD on Testnet



tryrlusd.com

Wallets? One SDK to rule them all: xrpl-connect



<https://github.com/XRPL-Commons/xrpl-connect>

XRPL Amendment Testing Suite



<https://tests.xrpl-commons.org/>

Scaffold-XRP



<https://github.com/XRPL-Commons/scaffold-xrp>

Bedrock



<https://github.com/XRPL-Commons/Bedrock>

Resources



<https://linktr.ee/rippledevrel>

